

Trabajo Práctico 04

CI: Pipelines as Code



Alumno	Marcos Don
Matrícula	2319274
Repositorio	github.com/marcosdon28/IngeSoft3-2319274
Workflow	.github/workflows/ci.yml
Jobs	build-backend · build-frontend (en paralelo)
Gate	required status checks + strict
Versión entregada	v4.0.0
Fecha	27/08/2026

1. Objetivo y qué se evalúa

Automatizar la verificación de la app: cada Pull Request y cada push a `main` construyen las dos imágenes, con el pipeline como **requisito obligatorio de merge**.

Requisito	Estado
Workflow en el repo, que entró por PR	✓ PR #18
Corre en cada PR y cada push a <code>main</code>	✓ 12 corridas registradas
Build de las dos imágenes con los Dockerfiles del TP2, en paralelo	✓ 1m53s en vez de 2m40s
Cache de capas funcionando	✓ 7 capas <code>CACHED</code> ; 1m53s → 21s
Required status checks activos sobre <code>main</code>	✓ los dos jobs, <code>strict: true</code>
Demostración del gate: rojo → bloqueado → fix → verde → merge	✓ PR #20
Status badge en el README	✓ PR #21

Qué NO hace este pipeline todavía, a propósito: no corre tests —eso es el TP5— y no publica las imágenes —eso es el TP7—. Hoy verifica que la app **se construye en una máquina limpia**, que ya es bastante más de lo que había.

2. Qué es CI, y qué no

Integración continua **no es «tener un pipeline»**: es la práctica de integrar el trabajo con frecuencia, verificando cada integración automáticamente. La herramienta es el medio.

El problema que ataca: **integrar tarde es integrar caro**. Si cada uno trabaja semanas en su rama, el día de la integración es un evento traumático. Si todos integran a diario y cada integración dispara la verificación, los problemas aparecen en minutos — cuando el cambio que los causó es chico y está fresco.

Se puede tener **pipeline sin CI**: un YAML que corre una vez por semana sobre ramas que viven meses. Y se puede tener **CI sin pipeline**, si todos integran a diario y verifican a mano — funciona hasta que el equipo crece o alguien se olvida. Lo que hace este TP es juntar las dos cosas: la práctica, y la máquina que la hace cumplir.

Pipeline as Code

El pipeline es un archivo **dentro del repo**, no una configuración clickeada en una web. Consecuencias: se versiona y se revisa como el código —un cambio al pipeline es un PR más, y de hecho el PR #22 de este práctico modifica el pipeline y **pasa por su propio gate**—, viaja con el código, y es reproducible.

Es la **cuarta aparición del mismo patrón** en la materia: todo lo importante se declara explícitamente en vez de hacerse a mano. TP1 las protecciones, TP2 el entorno, TP3 el plan enlazado al código, y ahora el proceso de build.

3. El workflow, decisión por decisión

```
name: CI

on:
  pull_request:
    branches: [main]      # el que hace el trabajo: verifica ANTES del merge
  push:
    branches: [main]      # deja la corrida que lee el badge, y su cache

jobs:
  build-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: docker/setup-buildx-action@v4
      - uses: docker/build-push-action@v7
      with:
        context: ./backend
        push: false
        tags: inventario-backend:ci
        cache-from: type=gha,scope=backend
        cache-to: type=gha,mode=max,scope=backend

  build-frontend:
    # ... idéntico, con context ./frontend y scope=frontend
```

Decisión	Por qué
Dos jobs, uno por imagen	Son artefactos independientes . Un job único daría un solo check en rojo sin decir cuál se rompió
En paralelo	Es el default de Actions y no hay dependencia que justifique serializarlos. El tiempo total pasa a ser el del más lento
<code>push: false</code>	Hoy sólo se verifica que la imagen se construya. Publicarla es el TP7
<code>scope</code> distinto por job	No es opcional con dos jobs: sin él comparten estante y se pisan
Versiones fijadas (<code>@v6</code> , no <code>@main</code>)	Sin versión fijada el pipeline cambiaría solo el día que sus autores publiquen algo

Lo que NO está en el archivo, y es lo más importante

No hay **una sola línea de Python ni de Node**. El workflow no sabe cómo se construye la app — eso lo sabe el Dockerfile del TP2. Si el pipeline compilara aparte, con `pip` y `npm` escritos en el YAML, habría **dos definiciones de build** que tarde o temprano divergen: estaría verificando una compilación distinta de la que después se despliega.

Los jobs no comparten filesystem. Cada uno corre en un runner limpio y distinto — son dos máquinas. Si un job necesitara algo que produjo otro, tendría que viajar como artefacto o declararse `needs:` para el orden.

4. El paralelismo, medido

Job	Duración
build-backend	1 m 53 s
build-frontend	47 s
Suma, si fueran secuenciales	2 m 40 s
Reloj real, en paralelo	1 m 53 s

Los dos jobs arrancaron en el **mismo instante** (12:45:20Z), así que el tiempo total es el del más lento y no la suma. El ahorro crece con cada job que se agregue: en el TP5 entran los tests.

El backend tarda más porque su etapa de build parte de `python:3.12` completa —1,62 GB, la que trae compilador— mientras que el frontend usa `node:22-alpine` . Es el costo del multi-stage del TP2, y se paga una sola vez gracias al cache.

5. El cache de capas

```
marco — IngeSoft3-2319274 — cache de capas: primera vs segunda corrida

marco@MacBook IngeSoft3-2319274 % gh run view 33073383889 --log | grep build-backend # PRIMERA corrida
#12 DONE 0.0s
#12 [build 3/4] COPY requirements.txt .
#13 DONE 10.6s
#13 [build 4/4] RUN python -m venv /opt/venv && /opt/venv/bin/pip install --u
#16 DONE 0.1s
#16 [final 5/5] RUN useradd --create-home --uid 1001 appuser
#17 DONE 66.6s
#9 importing cache manifest from gha:4393533581172612553

marco@MacBook IngeSoft3-2319274 % gh run view 33073929232 --log | grep build-backend # SEGUNDA corrida
#10 CACHED
#11 CACHED
#12 CACHED
#13 CACHED
#14 CACHED
#15 CACHED
#16 CACHED
#9 importing cache manifest from gha:5194117632613497586

marco@MacBook IngeSoft3-2319274 % gh api .../jobs --jq 'duracion de build-backend'
corrida 33073383889 build-backend: 1m 53s
corrida 33073929232 build-backend: 0m 21s

# Que cambio entre las dos: en la primera, el paso #13 (crear el venv e
# instalar las dependencias) se ejecuto de verdad; en la segunda salio del
# cache junto con otras seis capas.
#
# 'importing cache manifest from gha:...' es cache-from trayendo las capas
# desde el cache de GitHub Actions. No es el Docker de mi maquina ni el del
# runner: el runner nace VACIO en cada corrida.
#
# Detalle honesto: la primera corrida es mas lenta de lo que seria SIN cache,
# porque ademas de construir tiene que EXPORTAR las capas (el paso #17 tardo
# 66.6s). El cache se paga una vez y se cobra en todas las corridas siguientes.
```

Las dos corridas, con las líneas extraídas de `gh run view --log`. En la primera, el paso que crea el venv e instala dependencias se ejecutó de verdad; en la segunda salió del cache junto con otras seis capas.

Capa	¿Se reutiliza?	Por qué
Las imágenes base	✓	No cambian nunca
<code>COPY requirements.txt + pip install</code>	✓	Sólo se rehace si cambia <code>requirements.txt</code>
<code>COPY app ./app</code>	✗ si toqué código	Es lo que cambia en casi todos los PRs
Las capas posteriores a una que cambió	✗	Docker invalida esa capa y todas las siguientes

Ese orden no es casual: es la razón por la que el Dockerfile del TP2 copia las dependencias **antes** que el código. Al revés, cada cambio de una línea reinstalaría todo.

Qué pasa si el cache desaparece

Nada grave: el pipeline funciona igual, sólo más lento. El cache es una **optimización**, no una dependencia — vuelve a construir todo desde cero y a repoblarlo. Es importante que sea así, porque el cache se desaloja solo: GitHub lo purga por antigüedad y por límite de tamaño. Un pipeline que *necesita* el cache para funcionar está roto.

Detalle honesto: la primera corrida es más lenta que una sin cache, porque además de construir tiene que **exportar** las capas — ese paso tardó 66,6 s. El cache se paga una vez y se cobra en todas las corridas siguientes.

6. El pipeline como gate

En el TP1 la protección exigía que todo entrara por Pull Request. Este TP le suma la verificación automática: **required status checks**. Hoy `main` exige dos condiciones, y hacen falta las dos.

```
gh api --method PUT "repos/{owner}/{repo}/branches/main/protection" --input - <<'EOF'
{
  "required_pull_request_reviews": { "required_approving_review_count": 0 },
  "required_status_checks": {
    "strict": true,
    "contexts": ["build-backend", "build-frontend"]
  },
  "enforce_admins": true,
  "restrictions": null
}
EOF
```

El **PUT** reescribe la protección entera, no le agrega una línea. Todo campo omitido vuelve a su default — por eso el JSON re-declara también lo del TP1 (cero approvals y `enforce_admins`). Después de aplicarlo leí la protección de vuelta para confirmar que no se había perdido nada.

Los `contexts` son el **id del job**. Si mañana le pusiera `name:` a un job, el gate quedaría esperando un check que ya no existe y bloquearía todo.

The screenshot shows a GitHub Pull Request (PR) titled "demo: el gate en acción — build roto a propósito #20". The PR is from user "marcosdon28" and aims to merge a commit into the `main` branch from the `feature/demo-gate` branch. The PR status is "Some checks were not successful" with "1 failing, 1 successful checks".

The failing check is "CI / build-frontend (pull_request)" which is "Failing after 30s" and is marked as "Required". The successful check is "CI / build-backend (pull_request)" which is "Successful in 56s" and is also marked as "Required".

Below the checks, there is a "Squash and merge" button which is disabled. A message states: "You can also merge this with the command line. [View command line instructions.](#)".

At the bottom of the PR, there is a "Add a comment" section with a text area and a "Comment" button. There is also a "Close pull request" button with a red warning icon.

The footer of the page includes the GitHub logo, the text "© 2026 GitHub, Inc.", and links for Terms, Privacy, Security, Status, Community, Docs, Contact, Manage cookies, and Do not share my personal information.

El gate actuando: los dos checks marcados **Required**, `build-frontend` en rojo, `build-backend` en verde, y el botón *Squash and merge* deshabilitado. Alcanza con un solo check en rojo.

Desde la consola dice lo mismo, sin ambigüedad:

```
$ gh pr merge 20 --squash  
X Pull request #20 is not mergeable: the base branch policy prohibits the merge.
```

Un detalle de vocabulario que confunde

Con el build roto, la API devolvía `mergeable: MERGEABLE` y `mergeStateStatus: BLOCKED`. **No es contradictorio**: `mergeable` habla de **conflictos de contenido** (no había ninguno) y `mergeStateStatus` habla de **política** (el gate). Son dos preguntas distintas.

7. La demostración: rojo → bloqueado → fix → verde

1. **Rojo**: un `import` a un archivo inexistente en `frontend/src/App.jsx`. `build-frontend` falla a los 30 s.
2. **Bloqueado**: botón de merge deshabilitado; la consola lo confirma.
3. **Fix**: un commit que saca el import. El pipeline **re-corre solo**.
4. **Verde** → `mergeStateStatus: CLEAN` → merge.

Por qué rompí el frontend y no el backend

Es una diferencia de stack que vale la pena entender. El frontend **se empaqueta** — Vite resuelve los imports al hacer el bundle—, así que un import roto lo tumba durante `npm run build`. El backend es **Python: no compila ni se empaqueta**, y su Dockerfile **nunca ejecuta el código** — sólo instala dependencias. Escribir `import estonoexiste` en un `.py` daría **verde igual**: para romperlo habría que romper la **dependencia**, con un paquete inexistente en `requirements.txt`. Desde el TP5, con los tests adentro, el código del backend sí va a poder romperlo.

`strict: true` : por qué hacen falta dos PRs para verlo

Con un solo PR abierto esto no se puede demostrar. Dejé un segundo PR abierto **antes** de mergear el de la rotura; al mergearlo, el otro quedó desactualizado.

The screenshot shows a GitHub Pull Request (PR) for the repository 'docs: muestra-del-freno'. The PR is titled 'docs: nota sobre el pipeline en el README #19' and is authored by 'marcosdon28'. It is currently in the 'Development' branch and is attempting to merge into the 'main' branch. The PR status is 'All checks have passed' (2 successful checks). However, the merge is blocked because 'This branch is out-of-date with the base branch'. The message states: 'Merge the latest changes from main into this branch. This merge commit will be associated with marcosdon28.' The 'Update branch' button is visible. The 'Squash and merge' button is also present. The PR is currently in progress, and the 'Convert to draft' button is visible. The PR has 1 participant. The 'Add a comment' section is visible, showing a text input field and a 'Close pull request' button. The footer of the page shows the GitHub logo and copyright information: '© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information'.

All checks have passed y aun así el merge deshabilitado: la rama está *out-of-date with the base branch*. Su verde se sacó contra un `main` que ya no existe.

Sin `strict`, ese PR podría entrar en verde habiendo sido verificado contra un `main` anterior — y romper `main` sin que ningún check lo haya visto venir.

8. Visibilidad: el badge y el historial

The screenshot shows the GitHub interface for the repository 'IngeSoft3-2319274' by user 'marcosdon28'. The repository is public and has 17 commits. The file list shows various folders and files, each with a commit message and a timestamp. The sidebar on the right contains information about the repository, including its description, README, activity, stars, forks, releases, packages, contributors, and languages.

File/Folder	Commit Message	Commit Hash	Time Ago
.github/workflows	ci: construye las dos imágenes en paralelo, con cache de ...	6dfe9cd	11 minutes ago
backend	feat: agrega la app del semestre contenerizada (TP2) (#7)		18 hours ago
frontend	demo: el gate en acción — build roto a propósito (#20)		3 minutes ago
img	docs: agrega decisiones e informe del TP3 (#17)		17 hours ago
informes	docs: agrega decisiones e informe del TP3 (#17)		17 hours ago
scripts	feat: agrega la app del semestre contenerizada (TP2) (#7)		18 hours ago
.env.example	feat: agrega la app del semestre contenerizada (TP2) (#7)		18 hours ago
.gitignore	chore: agrega .gitignore base y CLAUDE.md con las reglas...		19 hours ago
CLAUDE.md	chore: agrega .gitignore base y CLAUDE.md con las reglas...		19 hours ago
README.md	docs: agrega el badge de CI al README (#21)		now
decisiones.md	docs: agrega decisiones e informe del TP3 (#17)		17 hours ago
docker-compose.registry.yml	feat: agrega la app del semestre contenerizada (TP2) (#7)		18 hours ago
docker-compose.yml	feat: agrega la app del semestre contenerizada (TP2) (#7)		18 hours ago
evidencias.md	docs: agrega evidencias del registry e informe del TP2 (#8)		18 hours ago

El badge en el README, mostrando el estado del último build de `main`.

```
[![CI](../actions/workflows/ci.yml/badge.svg)](../actions/workflows/ci.yml)
```

Son dos direcciones, y por eso los corchetes de afuera. La de adentro es la **imagen**; la de afuera es **adónde lleva al hacerle clic**. Con sólo la imagen el badge se ve igual, pero al clickearlo se abre el SVG suelto: una página en blanco. Es el error más fácil de cometer acá porque no se nota mirando el README.

El badge lee el **último build de main** — por eso el workflow necesita el trigger `push: branches: [main]`: con sólo `pull_request` no tendría de dónde leer.

marcosdon28 / IngeSoft3-2319274

Q Type to search

<> Code Issues 4 Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

Actions

New workflow

All workflows

CI

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

CI

ci.yml

Filter workflow runs

Help us improve GitHub Actions

12 workflow runs

	Event	Status	Branch	Actor
docs: agrega el badge de CI al README (#21)	main	now In progress		...
docs: agrega el badge de CI al README	docs/badge-ci	1 minute ago 18s		...
docs: nota sobre el pipeline en el README (#19)	main	1 minute ago 31s		...
docs: nota sobre el pipeline en el README	docs/muestra-del-freno	2 minutes ago 27s		...
demo: el gate en acción — build roto a propósito (#20)	main	3 minutes ago 36s		...
demo: el gate en acción — build roto a propósito	feature/demo-gate	7 minutes ago 40s		...
demo: el gate en acción — build roto a propósito	feature/demo-gate	10 minutes ago 59s		...

El historial completo: la corrida en rojo del build roto, el fix en verde, y el merge a `main`. Se ven los dos tipos de evento, `pull_request` y `push`.

Parece cosmético y es cultura: hace visible el estado del proyecto a cualquiera que entre al repo. Un badge en rojo, a la vista, es una presión sana para arreglarlo.

9. Problemas encontrados y aprendizajes

Estaba usando versiones viejas de las actions

Empecé con `checkout@v4`, `setup-buildx@v3` y `build-push@v6` — dos o tres majors por detrás. **Funcionaban**, así que el pipeline en verde no me lo iba a avisar nunca. Lo detecté consultando las releases de cada repositorio, y las subí en un PR aparte que pasó por su propio gate. Un pipeline en verde dice que algo *funciona*, no que esté *bien*.

Buscar `CACHED` por la API no funciona

`gh api .../actions/jobs/<id>/logs` devuelve una redirección a un zip, así que el `grep` daba cero y parecía que el cache no actuaba — cuando los tiempos decían lo contrario. El comando correcto es `gh run view <run> --log`. Antes de crearle a una medición conviene entender qué devuelve el comando que la produjo.

El contexto en los Pull Requests

En un PR, `GITHUB_REF_NAME` no vale el nombre de la rama sino `<numero>/merge`: GitHub construye una mezcla de la rama con `main` y verifica **eso**. Para loguear la rama real hay que usar `github.head_ref`. No es cosmético — aclara que lo que el pipeline verifica no es tu rama, sino el **resultado propuesto del merge**.

Declaración de uso de Inteligencia Artificial

Qué fue asistido	Cómo lo verifiqué
El <code>ci.yml</code>	Lo leí antes de commitarlo y sobre todo lo vi funcionar : los dos jobs arrancando en el mismo instante, 7 capas reutilizadas, y el gate frenando un merge de verdad
La configuración del gate por API	Leí la protección de vuelta después del <code>PUT</code> para confirmar que no se había perdido nada del TP1
Las mediciones	Salen de <code>gh api .../jobs</code> y <code>gh run view --log</code> , no de estimaciones

Un error que cometí y corregí, porque viene al caso. Al armar la evidencia del cache, la primera versión de la imagen tenía líneas de log de la primera corrida que **no había leído**: estaban escritas a mano para ilustrar. Lo detecté al releerla, la descarté entera y la rehíce extrayendo cada línea de `gh run view --log` de las dos corridas reales. **Una evidencia inventada es peor que no tener evidencia**: la segunda se nota, la primera no.

Lo que **no** delegué: la estructura del pipeline, la decisión de construir con el Dockerfile en vez de compilar aparte, qué romper para demostrar el gate, y la interpretación de las mediciones.

Anexo — Los comandos, en orden

```
# — El workflow, por PR como todo —————
git switch -c feature/ci-build-imagenes
# ... escribir .github/workflows/ci.yml ...
gh pr create --title 'ci: construye las dos imágenes en paralelo, con cache'
gh pr merge 18 --squash --delete-branch

# — El gate (OJO: el PUT reescribe TODA la protección) —————
gh api "repos/{owner}/{repo}/branches/main/protection" # mirar ANTES
gh api --method PUT "repos/{owner}/{repo}/branches/main/protection" --input - <<'EOF'
{ "required_pull_request_reviews": { "required_approving_review_count": 0 },
  "required_status_checks": { "strict": true,
                              "contexts": ["build-backend","build-frontend"] },
  "enforce_admins": true, "restrictions": null }
EOF
gh api "repos/{owner}/{repo}/branches/main/protection" --jq '.required_status_checks'

# — Segundo PR abierto ANTES, para poder ver strict: true —————
git switch -c docs/muestra-del-freno # ... cambio cualquiera ... gh pr create

# — La demostración del gate —————
git switch -c feature/demo-gate
# ... import a un archivo inexistente en App.jsx ...
docker build ./frontend # comprobar que falla también local
gh pr create --fill
gh pr merge 20 --squash # → "the base branch policy prohibits the merge"
# ... sacar el import ...
git commit -am 'fix: saca el import al archivo que no existe' && git push
gh pr checks 20 # → los dos en SUCCESS
gh pr merge 20 --squash --delete-branch

# — strict: true en acción —————
gh pr view 19 --json mergeStateStatus # → BEHIND, con los checks en verde
gh pr update-branch 19 # el pipeline re-corre sobre la mezcla
gh pr merge 19 --squash --delete-branch

# — El badge —————
git switch -c docs/badge-ci # ... la línea en el README ... gh pr create

# — Medir lo que se afirma —————
gh api "repos/{owner}/{repo}/actions/runs/<run>/jobs" --jq '.jobs[].started_at'
gh run view <run> --log | grep -E 'importing cache|CACHED'

# — Cerrar el práctico —————
git tag -a v4.0.0 -m "TP4 cerrado" && git push origin v4.0.0
```